

It has controllers for:

- Auth (login/register)
 - Admin
 - Cart
 - Products
 - Categories
 - Orders
 - User
-

1. AuthController (Login & Register)

 Base: /api/auth

 Purpose: User authentication

- POST /api/auth/register → Register new user
 - POST /api/auth/login → Login user (returns JWT)
-

2. AdminController (Admin operations)

 Base: /api/admin

 Purpose: Admin control panel

- GET /dashboard → Dashboard stats
 - POST /create-user → Admin creates user
 - POST /products → Add product
 - PUT /products/{id} → Update product
 - DELETE /products/{id} → Delete product
 - GET /orders → Get all orders
 - PUT /orders/{id}/status → Update order status
-

3. CartController (User cart)

👉 Base: /api/cart

✅ Purpose: Cart management

- GET /api/cart → Get cart
 - POST /api/cart/add → Add item
 - PUT /api/cart/update → Update quantity
 - DELETE /api/cart/remove/{id} → Remove item
 - DELETE /api/cart/clear → Clear cart
-

📁 4. CategoryController

👉 Base: /api/categories

✅ Purpose: Category data

- GET /api/categories → All categories
 - GET /api/categories/{id} → Category by ID
-

🛒 5. ProductController

👉 Base: /api/products

✅ Purpose: Product browsing

- GET /api/products → All products
 - GET /api/products/{id} → Product by ID
 - GET /api/products/featured → Featured products
 - GET /api/products/search?q= → Search products
 - GET /api/products/category/{categoryId} → Products by category
-

📦 6. OrderController

👉 Base: /api/orders

✅ Purpose: Order handling

- POST /api/orders → Place order

- GET /api/orders/my-orders → User orders
-

7. UserController

 Base: /api/user

 Purpose: User info

- GET /api/user/profile → User profile

My project follows layered architecture with Controller, Service, Repository, and Entity layers. Controllers handle HTTP requests, Services contain business logic, Repositories interact with the database, and Entities represent database tables.

DTOs are used to transfer data between layers, and Spring Security with JWT is used for authentication and authorization.”

Data seeder is used to insert default data into database automatically when spring boot application starts.

“I implemented JWT-based authentication using JwtService and JwtFilter. JwtService is responsible for generating tokens during login and validating them. JwtFilter runs on every request, extracts the token from the Authorization header, validates it using JwtService, and sets the user authentication in the SecurityContext. This ensures secure and stateless authentication across all APIs.”

Data seeder is used to insert default data into database automatically when spring boot application starts.

SecurityConfig is used to manage security in the application. It decides which APIs are public and which need authentication, and it also supports JWT and role-based access.”

DTO:

Data Object model

It is used to transfer data between layers or between client and server

*DTO is used to send only required data instead of exposing complete entity data

Entity:

```
class User {
```

```
    String name;
```

```
    String email;
```

```
String password;
}
```

DTO:

```
class UserDTO {
    String name;
    String email;
}
```

Keyword	Meaning
@JoinColumn	Creates foreign key column in table
mappedBy	Tells relation is handled on other side
OneToOne	One record ↔ one record
OneToMany	One record → many records
ManyToOne	Many records → one record
Cascade.ALL	Parent action affects child (save/delete)
orphanRemoval	Remove child from DB if removed from list

User → Cart → CartItems → Product

User → Order → OrderItems → Product

Product → Category

User —(1-1)—▶ Cart —(1-M)—▶ CartItems —(M-1)—▶ Product

User —(1-M)—▶ Order —(1-M)—▶ OrderItems —(M-1)—▶ Product

Product —(M-1)—▶ Category

Enum is special data used to define a group of constant values that cannot change.

@Enumerated(EnumType.STRING)

This is a **JPA annotation** used with enums

It tells Spring/JPA how to store enum values in the database